What Play Will Come Next? Predicting NFL Run vs Pass Using Machine Learning

Okema Gant ITCS 5154: Applied Machine Learning Spring 2026

Introduction

In football, if a defense knows whether the offense is about to run or pass before the snap, that is a massive advantage. This project is about trying to make that prediction using machine learning, specifically using pre snap game situation stuff like the down, yards to go, field position, and score.

Play calling in the NFL is one of the most argued about things in sports. Remember Super Bowl 49? Seattle was one yard away from winning with Marshawn Lynch right there, and they threw a pass that got picked off. Everyone and their mom had an opinion on that. So I wanted to see what the data says about how predictable these play calls really are.

Cameron Taylor did a Stanford CS230 project where he combined TV broadcast screenshots with game stats and used CNNs and transfer learning to predict plays. The surprising thing was the images did not really help at all. A regular Random Forest on just the numbers did just as well or better than the deep learning stuff. That is what got me interested in this topic. On top of that, building and training CNNs with image data would have taken way more time and computing resources than I had for this project, and since it did not even improve the results in Taylor's work, it did not make sense to go down that road given the scope of a semester project. So I focused on how far simple, interpretable models could go with just the game situation data.

The basic idea is: take 8 seasons of NFL play by play data (2009 to 2016), clean it up, pull out useful features from the game context, and train logistic regression and random forest models to predict run or pass. I also built an interactive play call advisor tool that recommends run or pass for any game scenario you give it.

Background

Related Work 1: Cameron Taylor, "Deep Learning for In-Game NFL Predictions" (Stanford CS230, Winter 2020)

This is the main paper I based my project on. Taylor used NFL play by play data from Kaggle (2009 to 2016 seasons) and also grabbed screenshots from TV broadcasts of actual games. He tried a bunch of approaches: a custom shallow CNN, a VGG19 transfer learning model, and traditional ML models like Random Forest and Lasso as benchmarks.

His thinking was that maybe the visual layout of the field, like how players are lined up, could give the model extra info that the stats alone do not capture. So he trained image based models on RGB screenshots along with the game stats.

Here is what he got: - Shallow CNN: around 60.6% accuracy on run vs pass - VGG19 Transfer Learning: about the same range - Random Forest (just the numbers): 61.4% accuracy - Lasso Regression: used for predicting yards gained

The big thing here is that the images did not really help. The Random Forest using just game situation stats matched or beat the CNN models. So basically the game context like down, distance, score, and field position already has most of the useful signal, and the broadcast images are just noise for this problem.

Pros: Really creative idea combining images and stats. He tested a lot of different model types which is cool. **Cons:** Collecting image data is a pain and not easy to scale up. The CNNs needed GPU time but did not beat simpler methods.

This paper is what motivated my project. Since the simple models worked just as well, and since building a CNN pipeline with image preprocessing would have been a huge time investment that was beyond the scope of what I could do in this class, I decided to skip the image stuff entirely and focus on making the tabular approach as clean and useful as possible.

Related Work 2: C. Joash Fernandes, R. Yakubov, Y. Li et al., "Predicting Plays in the National Football League" (Journal of Sports Analytics, 2020)

This paper is about the same thing I was working on, predicting pass vs rush in the NFL. They used data from the 2013 to 2017 regular seasons, so about 1,034 games and 130,344 plays. They wanted to build something that coaches and players could actually use in real time.

They tried a bunch of models and their best one was a neural network that got 75.3% accuracy. That is a lot higher than what Taylor or I got, but they used a different dataset with fewer seasons and a different time period, so it is hard to compare directly. They probably had different features too.

Pros: Got really good accuracy numbers. Focused on making it useful, not just technically cool. It is published in a real journal so the methods were checked. **Cons:** Only covers 4 seasons compared to my 8, so the patterns might not generalize as well. And the neural network that did the best is also the hardest one to explain.

Related Work 3: Roman Lutz, "NFLPlayPrediction" (GitHub Project)

This is a GitHub project by Roman Lutz that went wider than just run vs pass. He tried to predict a bunch of different things like touchdowns, yards gained, and first down conversions using NFL play by play data.

He threw a lot of models at the problem. SVMs, K Nearest Neighbors, Decision Trees, Logistic Regression, Neural Networks, and for regression stuff like predicting yards he used SVR and Linear Regression. His features were things like team, opponent, quarter, time left, field position, down, yards to go, formation, and play type.

Pros: Tested a ton of different models which is useful to see. Predicting multiple things instead of just play type gives you more info. PCA is a nice touch. **Cons:** Since it is just a GitHub repo and not a published paper, the documentation is not as detailed. Also his prediction targets are different from mine so it is tough to do a direct comparison.

Methods

Data

The data is from 8 seasons of NFL play by play records (2009 to 2016), pulled from Kaggle and nflscrapR. After I filtered it down to just run and pass plays and dropped rows that were missing important info, I ended up with about 370,000 plays. The split is roughly 55% pass and 45% run, which is close enough to even that accuracy works fine as the main metric.

Data Preprocessing

This part of the project ties directly back to Module 2 on data preprocessing.

Here is what the preprocessing does:

1. **Filtering**: I only kept run and pass plays. Anything like special teams, penalties, or weird plays got removed.
2. **Missing values**: If a row was missing something important like down, distance, yard line, or scores, I just dropped it. With 370K plays to work with, dropping was the simplest approach.
3. **Feature encoding**: Categorical stuff like team names, quarter, down, and drive number got turned into numbers. For Lasso I used one hot encoding, and for Random Forest regular integer encoding is fine since trees do not care about that.
4. **Feature scaling**: I used StandardScaler to normalize the numeric features so that logistic regression and Lasso would train properly.
5. **Train/Dev/Test split**: 75% for training, 12.5% for tuning (dev set), and 12.5% held out for final testing. This comes from what we learned in Module 5 about proper evaluation setup.

Features

All the features are things you would know before the snap:

- Quarter (1 through 5, where 5 is overtime)
- Down (1st through 4th)
- Yards to go for a first down
- Yard line (distance from opponent end zone, 1 to 99)
- Offensive team score
- Defensive team score
- Score differential (offensive score minus defensive score)

- Drive number
- Offensive team ID
- Defensive team ID

Nothing about what actually happened on the play is in the features, so there is no data leakage.

Models

Naive Baseline (54.6% accuracy): Just always guess "Pass" since pass plays happen a little more often. This is the bare minimum any real model needs to beat.

Logistic Regression (56.8% accuracy): A technique from Module 4 on linear classification. It is a simple linear model that weights each feature and spits out a probability. Above 50% means predict pass. It is basic but it shows there is some actual signal in the features beyond just the class split.

Random Forest (61.4% accuracy): This comes from Module 6 on tree based models. A Random Forest is basically a bunch of decision trees (hundreds of them) each trained on random subsets of the data. They all vote and the majority wins. This picks up on non linear stuff that logistic regression cannot see. Like how the relationship between field position and yards to go changes depending on the down, Random Forest figures those patterns out.

For the play call advisor tool, I used four models together: - A Random Forest Regressor for expected yards on run plays - A Random Forest Regressor for expected yards on pass plays - A Random Forest Classifier for first down probability on run plays - A Random Forest Classifier for first down probability on pass plays

These four models feed into a scoring function that combines expected yards and first down probability (first down matters more in short yardage) to give a final run or pass recommendation.

Overall Framework

Here is how everything flows:

1. Load the raw NFL play by play CSV files
2. Filter and clean the data
3. Build features from the game situation columns
4. Train models on the training set, tune on the dev set

5. Test on the held out test set for final numbers
6. The interactive advisor takes whatever game scenario you type in, runs it through the models, and tells you whether to run or pass with all the supporting numbers

Hyperparameter Tuning

Also from Module 5, hyperparameter tuning is about trying different model configurations and picking the one that performs best instead of just going with defaults.

For Lasso, LassoCV automatically tunes over 50 alpha values using cross validation.

For Random Forest, a grid search is performed over 65 combinations of: - Number of estimators: 1, 2, 3, 5, 7, 10, 13, 19, 26, 37, 50, 70, 100 - Max depth: 1, 3, 5, 10, 20

The best combination is selected based on dev set performance.

Experiments

Reproducing the Original Paper

Cameron Taylor's paper got 61.4% accuracy with Random Forest on run vs pass using this same Kaggle dataset, and I got the exact same number. Same data, same model type, same kind of features (down, distance, field position, scores, team IDs), so it makes sense the results match up.

The one thing I did not do was the CNN and VGG19 image models. We covered CNNs in Module 10 so I understand how they work, but Taylor's CNN models only got around 60.6% which is actually a bit worse than the Random Forest. So not only did the images not help, but setting up that whole pipeline with collecting screenshots, preprocessing images, and training deep learning models would have been a massive time sink. For a semester long project with limited time and no dedicated GPU setup, it just was not realistic or worth it when the simpler approach already matched the results.

My Experiments

On top of reproducing the paper, I did some extra stuff:

Logistic Regression as a simpler baseline: Applying linear classification from Module 4, I wanted to see how a basic linear model would do. It got 56.8%, which beats just guessing pass every time (54.6%) but is way behind Random Forest (61.4%). So there is definitely some nonlinear stuff going on in the data that trees pick up on.

Lasso Regression for yards prediction: Using Lasso from Module 3 on linear regression, I tried predicting yards gained instead of play type. Best alpha was 0.079, and 30 out of all the coefficients ended up non zero, which tells you roughly how many features actually matter since Lasso pushes the unimportant ones to zero.

Interactive Play Call Advisor: This is the part I am most proud of. It goes past just classifying plays. You give it a game scenario and it predicts expected yards and first down probability for both run and pass, then tells you which one to go with. For example, say Carolina is playing New England in Q3, 2nd and 7 from the opponent 65 yard line with the score tied at 14:

Metric	Run	Pass
Expected Yards	+4.69	+6.40

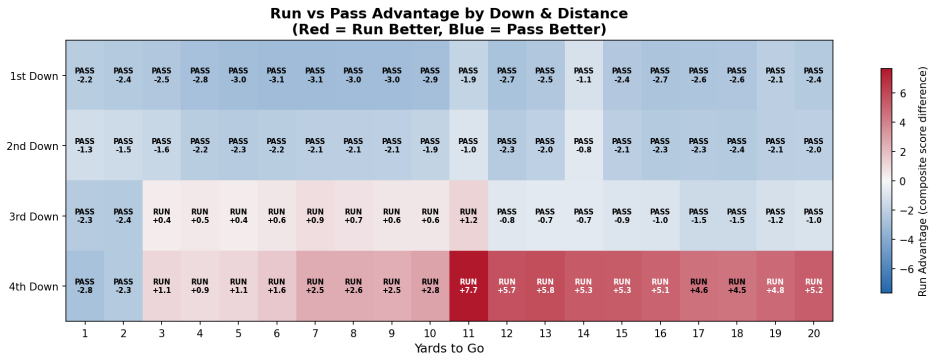
1st Down Probability	28.0%	41.6%
Composite Score	5.53	7.65

The model says pass and it is not even close.

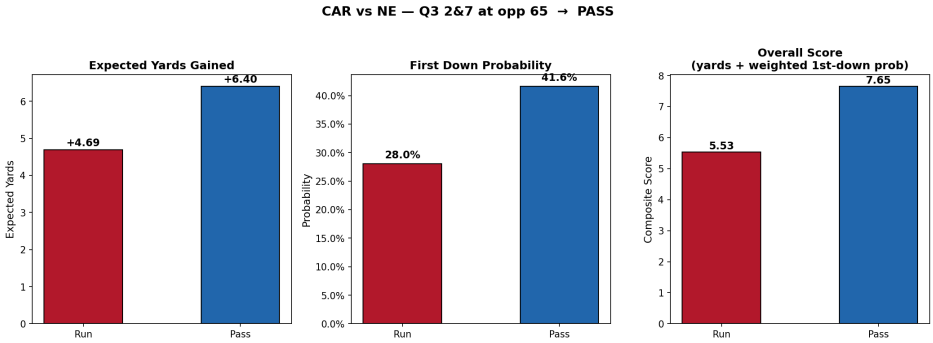
Visualizations and Analysis

I made five plots to dig into what the model is actually doing:

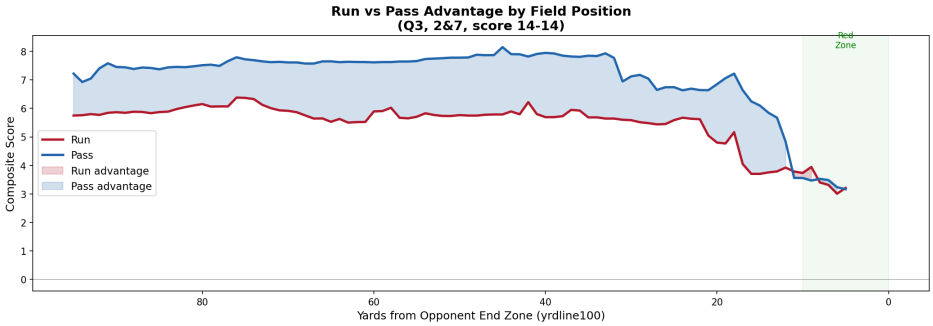
1. **Down and Distance Heatmap**: On 1st and 2nd down, passing is almost always the better call. 3rd and short is where running starts to compete. 4th and long actually favors running, which probably reflects teams in desperate situations making different choices. A heatmap works best here because you have two categorical variables (down and distance) and you want to see the pattern across every combination at once. Color coding makes it easy to spot trends without reading a bunch of numbers.



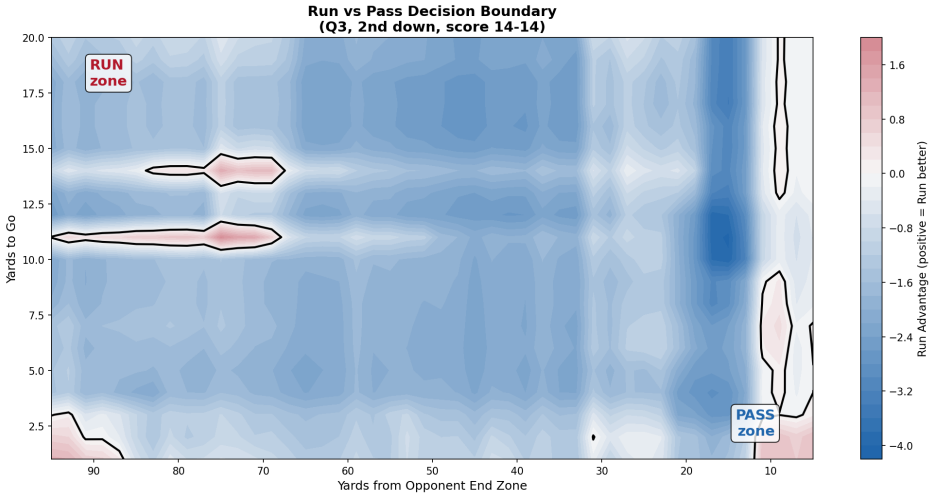
1. **Bar Chart Comparison**: Just a clean side by side of expected yards and first down probability for a sample scenario so you can see the difference at a glance. A bar chart is the right pick for this because you are comparing two options (run vs pass) across a couple of metrics. It keeps things simple and you can immediately see which bar is taller.



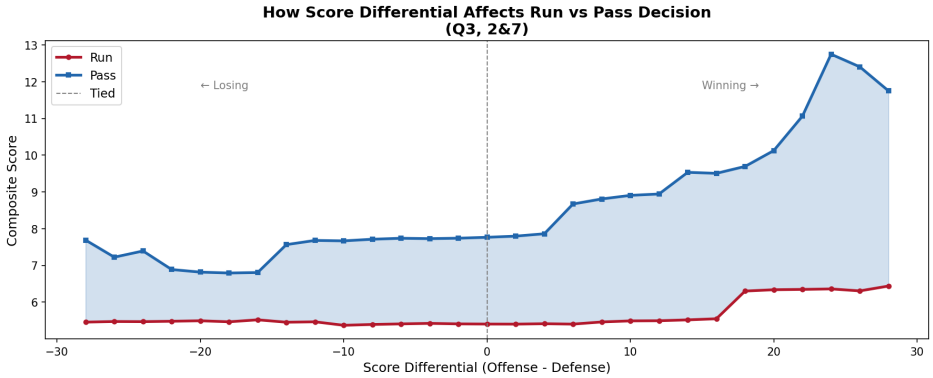
1. **Field Position Analysis**: The closer you get to the end zone, the less passing dominates and running becomes a real option. Makes sense because the field gets compressed in the red zone. A line plot works well here because field position is a continuous variable, so you want to see how the recommendation changes smoothly as teams move down the field rather than jumping between categories.



1. **Decision Boundary Contour**: A 2D plot with yard line on one axis and yards to go on the other. Red zones mean run, blue means pass. You can see exactly where the tipping points are. A contour plot is ideal for this because it shows the decision boundary across two continuous variables at once. You get a full map of where the model flips from one recommendation to the other, which would be impossible to show with a bar chart or table.



1. **Score Differential**: When you are winning big, passing advantage actually gets bigger. When you are behind, running gets a bit more competitive. That makes sense if you think about how games actually flow. A line chart fits this because score differential is a continuous range from negative to positive, and you want to see the trend of how the recommendation shifts as the score gap changes.



Discussion

The fact that a Random Forest at 61.4% can match deep learning models that had both images and stats is pretty telling. It means the game situation numbers already have most of the useful signal, and throwing broadcast screenshots at a CNN does not really add anything.

Now, 61.4% is not some crazy high number, but honestly I do not think you can get that much higher on this problem. Football is supposed to be unpredictable. Good offensive coordinators are literally trying to make their play calling hard to guess. If a team was perfectly predictable, defenses would destroy them. So there is kind of a built in ceiling here.

The cool thing is the patterns the model picks up actually make football sense. Short yardage? Run it. Losing? Pass more. In the red zone? Running becomes a bigger part of the plan. None of that is shocking if you watch football, but the model figured it all out just from the numbers.

Conclusions

So the big question this project was really trying to answer is: can machine learning actually predict whether an NFL team is going to run or pass? The honest answer is kind of, but not amazingly well, and there are good reasons for that.

The best model I built (Random Forest) got 61.4% accuracy. That beats just guessing pass every time (54.6%), so the model is learning something real. It picks up on patterns that make total football sense, like teams run more in short yardage, pass more when they are behind, and shift toward running in the red zone. The Fernandes paper even got up to 75.3% with a neural network on a different dataset. So ML can definitely find signal in play calling data.

But there is a ceiling on this problem that no model is probably going to break through. Football play calling is supposed to be unpredictable. That is literally the point. If an offensive coordinator was calling plays in a way that a computer could predict 90% of the time, defenses would eat them alive. Good play callers mix things up on purpose to keep defenses guessing. So in a way, the fact that models top out around 60 to 75% accuracy is not a failure of the models. It is just the nature of football.

That said, even at 61.4%, there is practical value here. If a defensive coordinator knows that in a specific game situation the offense runs 60% of the time instead of 50%, that is useful information for game planning. The play call advisor I built takes this further by not just predicting the play type but also estimating expected yards and first down probability for each option. So even if you cannot perfectly predict what is coming, you can understand the tendencies and plan around them.

On the modeling side, one thing that became clear is that you do not need complex deep learning to get these results. Simple tabular features with a Random Forest matched what CNNs trained on broadcast images could do. Given the time and scope of this class project, focusing on the simpler approach was the right call, and it turns out it was the right approach for the problem too.

If I had more time, here is where I would take this next:

The biggest change would be narrowing the focus. Right now the model looks at the entire NFL all at once, every team, every coordinator, every play style lumped together. But in reality, a defense is not preparing for the whole league. They are preparing for one opponent that week. So the next step would be building team specific models that learn the tendencies of a single offense. Every offensive coordinator has habits. Some guys love to run on first down, some are

pass heavy in the red zone, some get predictable when they are trailing. A model trained specifically on one coordinator's play calling history would probably be way more accurate than a league wide model because it is learning one person's decision making patterns instead of averaging across 32 different philosophies.

The idea would be to take this from a general prediction tool to something a defensive coordinator could actually use during game week. You feed it the opponent's play by play data from the season, the model learns that coordinator's tendencies in different situations, and then the defense can use those predictions to put themselves in better positions. If the model says this coordinator runs 70% of the time on 2nd and 2 inside the 10, the defensive coordinator can load up the box in that spot. It would not replace film study, but it could highlight patterns that are hard to catch just watching tape.

On top of that, building a simple web based UI would make it way more usable. Right now everything runs in a terminal, which is fine for a class project but nobody on a coaching staff is going to use that. A clean interface where you plug in the game situation and get a recommendation back would make this something people could actually use during a game or during the week while game planning.

Sharing Agreement

No, I do not agree to share this work.

References

Taylor, C. (2020). "Deep Learning for In-Game NFL Predictions." Stanford CS230: Deep Learning, Winter 2020. Available at: https://cs230.stanford.edu/projects_winter_2020/reports/32263160.pdf

Horowitz, M. et al. "NFL Play-by-Play Data 2009-2016." Kaggle. Available at: https://www.kaggle.com/datasets/maxhorowitz/nflplaybyplay2009to2016

Fernandes, C. J., Yakubov, R., Li, Y. et al. (2020). "Predicting Plays in the National Football League." Journal of Sports Analytics, 6(1), 35-43. DOI: 10.3233/JSA-190348

Lutz, R. "NFLPlayPrediction." GitHub Repository. Available at: https://github.com/romanlutz/NFLPlayPrediction

Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825-2830.